A ChaRNABERT-style model for human DNA

GBST tokenization + bidirectional BERT encoder, on the DNA-xLSTM code base

Progress report · Omar R.A Mohamed · 4 October 2026, 18:30 CEST

Code: https://github.com/OMAR622-ai/dna-xlstm-charna (branch `charna-gbst`, private)

Summary. The ChaRNABERT architecture — a learnable tokenizer (GBST) in front of a bidirectional BERT encoder — was implemented in the DNA-xLSTM code base, tested (23 automatic tests pass) and adapted to human DNA (hg38). It is being pretrained on 0.5 billion bases, next to a same-size comparison model without GBST (Transformer++). First result: the validation loss of GBST-BERT fell from 1.340 (0.1 B bases) to 1.301 (0.2 B bases); random guessing is 1.386. The accuracy table and loss curves of both models are expected tonight, 4 October, and will be added to an updated version of this report. No benchmark (downstream task) results yet.

Contents

1 Goal 1

2 How the ChaRNABERT model works 1

3 How we apply ChaRNABERT to DNA 3

4 What was done, step by step 3

5 Results so far 4

6 Differences from the papers 5

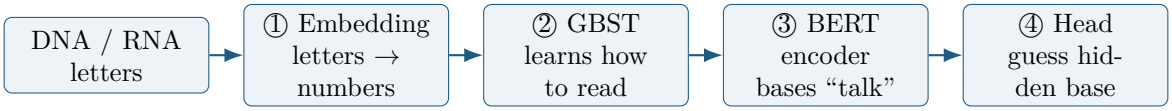
7 Next steps 6

1 Goal

The project asks whether a *learned* way of reading DNA helps a small model. ChaRNABERT (Morales-Pastor et al., 2024) showed this for RNA. We take its architecture, train it on the human genome with the short-context protocol of the DNA-xLSTM paper (Schmidinger et al., 2024), and compare it with models from that paper. The plan has three parts: (1) pretraining on hg38 with masked language modelling; (2) evaluation on held-out DNA (accuracy and loss); (3) later, the 18-task Nucleotide Transformer (NT) benchmark. Parts (1)–(2) are running; part (3) is ready but postponed.

2 How the ChaRNABERT model works

ChaRNABERT is an RNA "foundation model" from Nostrum Biodiscovery (arXiv:2411.11808). Its name combines *Cha*racter (single letters), *RNA* and *BERT*. Its central idea: most models fix in advance how a sequence is cut into "words" (single bases, codons, k-mers); ChaRNABERT lets the model **learn how to cut, separately at every position**.



① Embedding — letters become numbers. Each letter is replaced by a learned list of numbers (256 in our model), like a profile describing that nucleotide.

② GBST (Gradient-Based Subsequence Tokenization) — learning how to read.

1. *Smooth*: each base is mixed a little with its neighbours (a short filter).
2. *List candidates*: at every position, every block of 1, 2, 3 or 4 bases that contains the position. Blocks of the same size start at different places, like reading frames ("offsets"). This gives 10 candidates per position (Table 1).
3. *Score*: a small learned scorer rates each candidate ("how useful is this reading here?"); the scores become weights that add up to 1.
4. *Make the choice consistent*: positions with similar weight patterns pull towards each other, so the "reading style" is consistent along the sequence (ChaRNABERT Eq. 6).
5. *Blend*: each base's output is the weighted mix of its 10 candidates. There is one output per base — ChaRNABERT removes Charformer's shortening step, because many tasks need single-base detail.

Biology analogy: a ribosome reads triplets, a transcription factor a 6–10-base motif, a methylase CpG pairs. GBST can learn to read "like a ribosome" in one place and "like a methylase" in another, without being told which.

Table 1: The 10 GBST candidates for position 3 (the first T) of `ACGTTGCA`.

Block size	Candidate blocks that contain position 3
1	`T`
2	`TT` (positions 3–4), `GT` (2–3)
3	`TTG` (3–5), `GTT` (2–4), `CGT` (1–3)
4	`TTGC` (3–6), `GTTG` (2–5), `CGTT` (1–4), `ACGT` (0–3)

③ BERT encoder — bases talk to each other. A stack of identical blocks (4 in our model). In each block, *attention* lets every base gather information from every other base, in both directions (*bidirectional*); then a small *feed-forward* network processes each position. The paper adds RoPE (encodes distances between bases), QKNorm (keeps training stable), SwiGLU (a better feed-forward type) and FlashAttention 2 (faster attention).

④ Prediction head. For every position, a score for each possible letter.

Learning: the fill-in-the-blank game. In *pretraining*, 15% of the bases are hidden and the model must guess them from the context (*masked language modelling*, MLM). To do this well it must learn motifs, codon patterns, pairing rules — without any labels. The paper also tested *UL2*, which additionally hides longer stretches. ChaRNABERT was trained on 62 million RNA sequences (31 M non-coding from RNAcentral, 31 M coding from RefSeq), with models of 8 M to 650 M parameters. Afterwards it is *fine-tuned* on specific labelled questions.

What the paper found. GBST gave a lower loss than single-letter reading in most settings (e.g. 0.498 vs. 0.508 for the 5 M model, Table 2); performance saturated around 30–50 M parameters; the 8 M model matched or beat much larger models on several tasks, e.g. RNA–protein binding F1 0.833 vs. 0.831 for the 650 M RiNALMo (Table 5).

3 How we apply ChaRNABERT to DNA

The model stays the same; the data and the setting change.

1. **DNA instead of RNA.** The human genome hg38 (about 3 billion letters), in pieces of 1,024 bases. Alphabet A, C, G, T (N = unknown, ignored).
2. **Letters to numbers.** A = 7, C = 8, G = 9, T = 10 (the repository's tokenizer).
3. **The same game.** 15% of the bases are hidden and must be guessed; with 50% chance the reverse-complement strand is shown instead, so the model learns that both strands are equivalent.

Real:	`A C G T T G C A A G T C`	
Input:	`A C ? T T G C A A ? T C`	
Answer:	`. . G . . . . . . G . .`	(dots: not hidden, not graded)

4. **The same architecture**, GBST + BERT encoder, but small: 2.1 million parameters, the size of the DNA-xLSTM paper's models, so the comparison is fair.
5. **Train, test, compare.** Train on millions of DNA pieces; test on DNA never seen in training (accuracy of the guesses); compare with the same encoder *without* GBST (Transformer++). If ours guesses better, GBST helps on DNA.

Table 2: ChaRNABERT paper vs. this project.

	ChaRNABERT paper	This project
Data	RNA (RNAcentral, RefSeq)	Human DNA (hg38)
Letters	A, C, G, U	A, C, G, T
Model	GBST + BERT	GBST + BERT (same)
Training game	Guess hidden bases (MLM; UL2 tested)	Same (MLM; UL2 available as an option)
Model size	8 M – 650 M parameters	2.1 M (to compare with DNA-xLSTM)
Piece length	up to 8,190 bases	1,024 bases (DNA-xLSTM protocol)
Later tasks	BEACON RNA benchmark	Nucleotide Transformer DNA benchmark

4 What was done, step by step

1. **Read the code base and the four papers** (Bio-xLSTM, ChaRNABERT, Charformer, UL2); every setting was taken from a paper table or section, or marked as our own choice.
2. **Server set-up.** JKU server `student01`: 4× NVIDIA GTX 1080 Ti, shared with another user. Installed a Python environment that works on these older GPUs (PyTorch 2.2 + CUDA 12.1); newer PyTorch versions no longer support them. Copied the genome (3.1 GB) and checked it with an md5 checksum. Set up key-based SSH login.
3. **Implemented the model** (`models/charna/`): GBST (`gbst.py`), the bidirectional encoder and prediction head (`modeling_charna.py`), settings (`configuration_charna.py`), and registered it in the training program. Existing models behave exactly as before.
4. **Implemented UL2 masking** as an option (`src/dataloaders/utils/ul2.py`).

5. **A launcher** (`scripts_charna/train_charna.py`) so the repository's training program runs on GPUs that cannot install two optional packages (mamba_ssm, flash_attn).
6. **Scripts**: pretraining (`run_pretrain_charna.sh`), benchmark fine-tuning, results table, loss plot, accuracy evaluation (`evaluate_mlm.py`), data download with checksum, a small genome sample for GitHub, result collection.
7. **Tests**: 23 automatic tests (Section 5); a check that the model can memorise one batch; a 3-step trial on 4 GPUs; a 48-step smoke run of both models.
8. **Matched the paper exactly** where possible (after review on 4 October): added the "pointwise" layer that ChaRNABERT's parameter table (Table 9) implies; switched the learning rate to ChaRNABERT's 5e-4 (Table 2); restored UL2 as an option.
9. **Bugs found and fixed**: (a) loading pretrained GBST weights into the benchmark code crashed; (b) the benchmark data loader mixed all 18 tasks together (461,850 examples instead of e.g. 5,509) — a bug in the original repository caused by a change on Hugging Face; (c) a results file wrote values under wrong column names; (d) the GitHub genome sample first held only 1/8 of each region; (e) smaller script issues (wrong exit status in a log, a path error).
10. **Small experiment** (running since 16:20 CEST): both models trained for 500 steps (≈0.5 billion bases); then accuracy evaluation on 2,048 held-out sequences. The 40-hour full run was stopped to get results sooner; the benchmark was postponed.
11. **Documentation and GitHub**: project README (front page), a beginner's explanation (`docs/EXPLAINED.md`), everything collected in one folder (`charna_project/`: code, data that fits in git, results), and this report.

Main settings (all for GBST-BERT; source in brackets):

Model	256 numbers per base, 4 encoder blocks, 8 attention heads, 2,107,392 parameters [Bio-xLSTM Table A3]
GBST	blocks of 1–4 bases with offsets, sum pooling, calibration, no shortening [ChaRNABERT §3.2]
Masking	15% of bases; of these 80% `[MASK]`, 10% random, 10% unchanged [Bio-xLSTM App. C.5]
Batch	1,024 sequences × 1,024 bases = 1,048,576 bases per step [Bio-xLSTM App. C.5]
Learning rate	5e-4 [ChaRNABERT Table 2]; warmup 10% of steps, cosine decay [Bio-xLSTM Table A4]
Optimizer	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1 [Bio-xLSTM Table A4]
Length of training	small experiment 500 steps (0.5 B bases); full run 10,000 steps (10.5 B)
Comparison model	Transformer++-2M, learning rate 1e-2 [Bio-xLSTM Table A3]

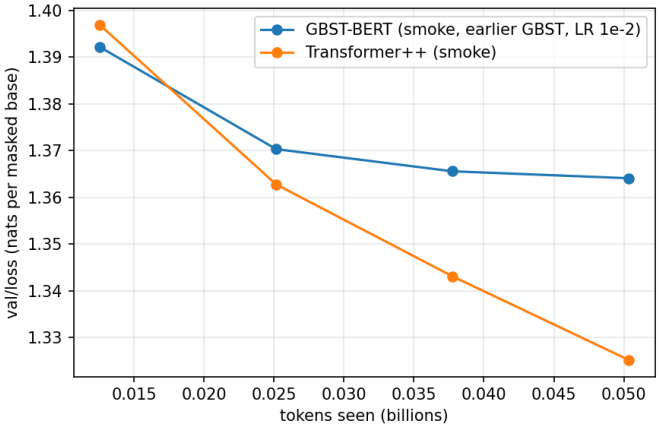
5 Results so far

Automatic tests: 23 of 23 pass. They check, among others, that GBST matches a slow reference implementation of the paper's equations, that it has the weight count of the paper's Table 9, that the model looks in both directions, that the loss counts only hidden bases, that about 15% of bases are hidden, that UL2 never leaks the answer, and that pretrained weights load correctly into the benchmark code.

Memorising one batch: passed. The loss fell below 0.05 after 164 steps (MLM) and 230 steps (UL2), so the model is able to learn.

Smoke run (48 steps, 50 million bases, an earlier GBST version with learning rate 1e-2). Loss is in nats per hidden base; 1.386 = random guessing among 4 bases.

Bases	GBST-BERT	Transformer++
13 M	1.392	1.397
25 M	1.370	1.363
38 M	1.366	1.343
50 M	1.364	1.325



Both models learn, but this run only checked that everything works: it was very short and used the older GBST and learning rate. A test of the accuracy script on these smoke models showed why accuracy needs reference values: the smoke GBST model reached 29.1% only by always answering "A" (the most common hidden base), i.e. no better than no model; Transformer++ reached 32.8%.

Small experiment (running). Current GBST-BERT validation loss (2% of the validation set):

Step	Bases seen	Validation loss	Test loss
100	0.10 B	1.3399	1.3437
200	0.21 B	1.3011	1.3013

The loss is falling clearly below the random-guess level (1.386) and below the smoke run. Still to come tonight: steps 300–500, the Transformer++ run, and the accuracy table (loss, accuracy, accuracy per base, compared with random guessing and with always guessing the most common base).

Not done yet: the full 10.5-billion-base training, the NT benchmark, and the planned comparisons (with/without GBST at full scale, MLM vs. UL2, with/without strand flipping).

6 Differences from the papers

Paper	This project	Reason
FlashAttention 2, bf16 numbers	standard attention, fp32	Our GTX 1080 Ti GPUs support neither; the result is the same, only slower
Eq. 3: offsets written as $X_{i+o:i+o+b}$	blocks that contain position i	Read literally, a position would get blocks that do not include it; the text (reading frames) describes our version
Eq. 9: SwiGLU written as $\sigma(xW_1) \odot \text{swish}(xW_2)$	standard SwiGLU	The text cites the standard SwiGLU; the printed formula is a different function
Eq. 1: depthwise convolution	+ a 256×256 pointwise layer	The paper's Table 9 counts $d \cdot M + d^2$ weights, which implies it
Maximum block size not stated	4	As in Charformer's figure (our choice)
RNA, 2–8 k bases, 8–650 M parameters	DNA, 1,024 bases, 2 M	Comparability with DNA-xLSTM (project plan)
Repository Transformer++ script: LR 8e-3, hidden size 1,024	LR 1e-2, hidden size 320	We follow the Bio-xLSTM paper's Table A3

Uncertain points: the block size (4); our reading of Eq. 3 and Eq. 9; the inferred pointwise layer; whether learning rate 5e-4 (best for ChaRNABERT's 5 M RNA model) is also good for a 2 M DNA model with much larger batches; the small experiment validates on only 2% of the validation data, so its curve is noisier than the full run's will be.

7 Next steps

1. Tonight: finish the small experiment; add the loss curves and the accuracy table to this report and to GitHub.
2. Review the results together; decide whether to start the full 40-hour training, and for which models.
3. After full training: the Nucleotide Transformer benchmark (18 tasks) and the comparisons.

References

- Morales-Pastor et al. (2024). Character-level Tokenizations as Powerful Inductive Biases for RNA Foundational Models (ChaRNABERT). arXiv:2411.11808.
- Schmidinger et al. (2024). Bio-xLSTM: Generative modeling, representation and in-context learning of biological and chemical sequences. arXiv:2411.04165.
- Tay et al. (2022). Charformer: Fast Character Transformers via Gradient-based Subword Tokenization. ICLR 2022, arXiv:2106.12672.
- Tay et al. (2023). UL2: Unifying Language Learning Paradigms. ICLR 2023, arXiv:2205.05131.
- Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.